

Università degli studi di Cagliari  
Corso di laurea in Data science, business analytics e innovazione  
Corso di laboratorio di Big Data  
Prof. Alessandro Giuliani

## PROGETTO FINALE DI LABORATORIO DI BIG DATA

Focus Tecnico: Costo delle operazioni Shuffle-Heavy in Dask

Candidato:  
Mirco Laconi

Matricola:  
11/82/00520

Anno accademico:  
2025/2026

# INDICE

## 1. Descrizione sintetica del problema

1.1 Obiettivo dell'analisi

1.2 Definizione della Variabile Target

1.3 Scelte Architetturelle e Framework

## 2. Analisi e descrizione del Dataset

2.1 Dataset utilizzati e impostazione del lavoro 2.2 Il corretto partizionamento

## 3. Data Cleaning e Validazione

3.1 Cast Tipologico e Validazione dei Formati

3.2 Imputazione dei Missing Values (Valori Nulli)

## 4. Costruzione della Tabella ML-Ready e Feature Engineering

4.1 Feature Engineering e Aggregazione

4.2 Definizione della Variabile Target e Prevenzione del Leakage 4.3 Pre-elaborazione per il Machine Learning (Pipeline Scikit-Learn)

## 5. Il costo delle operazioni Shuffle-Heavy

5.1 Pipeline A: Aggregazione Locale e Tree Reduction

5.2 Pipeline B: Rallentamento di rete e Shuffle forzato

5.3 Profilazione Visiva tramite Task Stream

5.4 Validazione dell'Integrità e Non-Determinismo Floating Point

## 6. Machine Learning Distribuito e Analisi dei Modelli

6.1 Scelta degli Iperparametri e Integrazione con Dask

6.2 Architettura Algoritmica e Tempi di Esecuzione

6.3 Analisi Critica delle Metriche Predittive

## 7. Limiti dell'Analisi e Sviluppi Futuri

7.1 Limiti Architetturelle e di Scala

7.2 Limiti Modellistici e Sviluppi Futuri

## 8. Conclusioni

# 1. Descrizione sintetica del problema

## 1.1 Obiettivo dell'analisi

Il presente elaborato descrive la progettazione, l'implementazione e la validazione di una pipeline completa di elaborazione dati in ambiente distribuito, sviluppata come progetto finale per il corso di Laboratorio di Big Data. L'obiettivo analitico del progetto consiste nel partire da un registro grezzo di eventi digitali per costruire un set di dati strutturato a livello di singola sessione utente, rendendolo pronto per algoritmi di Machine Learning (ML-ready). Nello specifico, il task di classificazione richiesto mira a prevedere se l'attività di navigazione di un utente all'interno di una medesima sessione coinvolga un solo comune oppure sia distribuita su più comuni. Tale problema richiede la transizione da un'analisi tabellare standard a un paradigma di calcolo parallelo, capace di aggregare milioni di record ottimizzando l'uso delle risorse hardware disponibili.

## 1.2 Definizione della Variabile Target

Per addestrare i modelli di classificazione, è stata ingegnerizzata un'apposita variabile target denominata `cross_municipality_session`. Come definito dalle specifiche di progetto, tale variabile assume un formato binario: Valore 1: se, aggregando gli eventi per `session_id`, la cardinalità dei comuni unici visitati (`nunique`) è strettamente maggiore di 1. Valore 0: se tutti gli eventi della medesima sessione afferiscono a un unico comune. Per garantire la validità scientifica dei modelli addestrati ed evitare il fenomeno del Data Leakage, il nome specifico del comune (o l'elenco dei comuni visitati) è stato rigorosamente escluso dall'insieme delle feature predittive, in quanto informazione direttamente correlata e causale rispetto alla variabile target.

## 1.3 Scelte Architetture e Framework

Per l'elaborazione del dataset è stato adottato Dask, un framework open-source nativo in Python per il calcolo parallelo e distribuito. La scelta di Dask è motivata dalla sua architettura a grafo ritardato (`lazy evaluation`), che permette sia di superare i limiti di memoria, tramite una suddivisione di dataset di grandi dimensioni in partizioni gestibili singolarmente dai worker, sfruttando meccanismi di `spilling` su disco in caso di saturazione della RAM; che di analizzare le performance di rete, sfruttando la Dask Dashboard per monitorare in tempo reale il Task Stream, i colli di bottiglia e l'overhead generato dal trasferimento dati. Questa infrastruttura si è rivelata essenziale per affrontare il Focus Tecnico del progetto, permettendo di misurare empiricamente e visivamente il costo computazionale delle operazioni `shuffle-heavy` rispetto a strategie di aggregazione locale.

# 2. Analisi e descrizione del Dataset

## 2.1 Dataset utilizzati e impostazione del lavoro

Il presente progetto si fonda sull'analisi di un registro di eventi digitali, strutturato in due tabelle principali fornite in tre formati di dimensione crescente (S, M, L). La prima tabella, denominata `events.csv`, contiene i log granulari delle interazioni utente, tracciando attributi

chiave quali l'identificativo della sessione, il comune di pertinenza, il tempo di permanenza sulla pagina e la profondità di scorrimento. A questa si affianca la tabella `municipality_catalog.csv`, consigliata dalle specifiche di progetto, utile a fornire il contesto anagrafico e le caratteristiche territoriali associate ai codici dei comuni. È opportuno precisare che, sebbene le specifiche di progetto indicassero come consigliato l'utilizzo del file `municipality_catalog.csv`, in fase di ingegnerizzazione si è scelto deliberatamente di escluderla dalla pipeline. La variabile `target_cross_municipality` richiedeva infatti un puro calcolo di cardinalità (conteggio dei comuni unici), operazione eseguibile direttamente sui codici della tabella `events`. Evitare un'inutile operazione di JOIN distribuita con il catalogo ha permesso di risparmiare un notevole overhead di rete, mantenendo il focus architetturale interamente sullo stress-test della funzione di `Shuffle S10`. Al fine di validare l'architettura logica e dimostrare la scalabilità della pipeline sviluppata, il processo di testing è stato articolato in tre fasi incrementalì. Le prime fasi di prototipazione, finalizzate all'impostazione delle regole di data cleaning e alla struttura delle aggregazioni, sono state condotte sulle taglie "S" (11.6 MB) e "M" (51.8 MB). Tuttavia, per evidenziare in modo inequivocabile i colli di bottiglia architetturali legati alle operazioni di rete, lo stress test definitivo è stato eseguito sul dataset di taglia "L", avente un peso su disco di circa 178.7 MB.

## 2.2 Il corretto partizionamento

L'esecuzione dell'analisi in un ambiente a singola macchina espone al rischio di un utilizzo indiscriminato della memoria centrale (RAM), un fattore che tende a mascherare le inefficienze computazionali del codice. Per simulare con rigore le dinamiche restrittive tipiche di un cluster distribuito, è stato istanziato un cluster locale Dask rigidamente configurato. L'infrastruttura è stata limitata a 4 worker attivi, ciascuno dotato di 2 thread logici, imponendo un tetto massimo di memoria pari a 1 GB per nodo. Questa precisa scelta ingegneristica costringe il sistema ad affrontare scenari controllati di saturazione della memoria (Out-of-Memory), innescando i naturali meccanismi di spilling su disco e forzando l'utilizzo della rete locale per lo scambio dei dati tra i nodi, permettendo così di misurare empiricamente i costi di trasferimento. In tale contesto, la strategia di ingestione dei dati assume un'importanza cruciale. Caricare in memoria un dataset distribuito delegando i parametri di default al framework genera frequentemente partizioni sbilanciate o eccessivamente gravose per i singoli worker, innescando il cosiddetto Straggler Effect. Per scongiurare questa eventualità e garantire un utilizzo ottimale degli 8 thread logici a disposizione, si è adottata una logica di Over-partitioning, tecnica che prevede il calcolo matematico della dimensione ideale del blocco di lettura (blocksize), con l'obiettivo di frammentare il file di input in un numero di partizioni pari o leggermente superiore alla disponibilità dei core di calcolo. Il dimensionamento del blocco è stato pertanto adattato in base alla mole del file elaborato. Ad esempio, per la transizione dalla taglia "M" alla taglia "L", si è passati dall'impostare un blocksize di 7 MB a un blocco calcolato di 23 MB. Tale parametrizzazione ha garantito l'estrazione di 8 partizioni perfettamente bilanciate dal file di 178.7 MB, assicurando che lo scheduler di Dask potesse distribuire equamente il carico, mantenendo la porzione di dati residente in RAM di ciascun worker abbondantemente al di sotto della soglia critica di 1 GB.

## 3. Data Cleaning e Validazione

Il processo di Data Cleaning rappresenta un passaggio fondamentale nell'Ingegneria dei Dati, specialmente in contesti distribuiti dove dati anomali possono causare errori irreversibili a cascata in fase di aggregazione. L'analisi preliminare dello schema nativo del dataset `events` ha

evidenziato diverse criticità legate sia alla coerenza dei tipi di dato (data typing) sia alla presenza di valori nulli o mancanti (missing values), che hanno richiesto interventi mirati tramite operazioni parallele.

### 3.1 Cast Tipologico e Validazione dei Formati

La prima anomalia rilevata riguarda l'ingestione delle variabili quantitative. Campi cruciali per l'analisi comportamentale, come il tempo di permanenza (`dwel_time_sec`) e la percentuale di scorrimento della pagina (`scroll_depth`), pur rappresentando grandezze numeriche (frazioni di secondo o percentuali), presentavano valori "sporchi" o caratteri non standard. Questo comportava che il lettore CSV di Pandas (e conseguentemente Dask) le interpretasse nativamente come semplici stringhe di testo (object), impedendo qualsiasi successiva operazione matematica come la somma o la media. Per risolvere questa problematica a livello distribuito, si è applicata la funzione `map_partitions` unitamente al metodo `pd.to_numeric` con parametro `errors='coerce'`. Questa tecnica forza la conversione tipologica dell'intera colonna da stringa a float, intercettando eventuali stringhe illeggibili o caratteri spuri e trasformandoli in valori NaN (Not a Number), preservando l'integrità matematica del formato senza far fallire il processo.

### 3.2 Imputazione dei Missing Values (Valori Nulli)

La forzatura tipologica sopra descritta, unita alla natura grezza dei log originali, ha inevitabilmente generato una quota di valori nulli all'interno del DataFrame. In vista della successiva modellazione di Machine Learning, tali assenze dovevano essere imputate in modo logico, applicando regole differenziate in base alla natura della variabile:

**Variabili Quantitative (`dwel_time_sec`, `scroll_depth`):** L'assenza di un tempo di permanenza o di un dato di scorrimento è stata interpretata semanticamente come un evento istantaneo o privo di interazione fisica (es. `bounceimmediato` dell'utente). Di conseguenza, i NaN presenti in queste colonne sono stati imputati con il valore numerico fisso 0.0, garantendo che l'aggregazione tramite somme o medie non producesse risultati distorti.

**Variabili Categorie (`device_type`, `referrer`, `content_category`):** Campi come il tipo di dispositivo o il canale di provenienza rappresentano le caratteristiche qualitative dell'evento. In mancanza di queste informazioni nel file sorgente, l'eliminazione delle righe (drop) avrebbe comportato una perdita di dati inaccettabile (alterando ad esempio il conteggio degli eventi totali per sessione). Pertanto, i valori nulli in queste colonne sono stati codificati introducendo una nuova classe convenzionale denominata "unknown" (sconosciuto). Tutte queste operazioni sono state strutturate come trasformazioni lazy all'interno del task graph di Dask, venendo eseguite fisicamente in memoria solo al momento del trigger dell'azione di aggregazione successiva, garantendo la massima efficienza prestazionale del cluster.

## 4. Costruzione della Tabella ML-Ready e Feature Engineering

Superata la fase di pulizia, il progetto ha affrontato una delle sfide centrali della preparazione dei dati: il cambio di granularità. Il dataset di partenza era strutturato "ad eventi", dove un singolo utente generava decine di righe diverse per ogni click o scorrimento di pagina, ma per poter addestrare un algoritmo di Machine Learning è necessario fornire una tabella "piatta", dove ogni singola riga rappresenta un'osservazione completa e indipendente. Come richiesto dalle

specifiche tecniche, abbiamo quindi dovuto comprimere l'intera storia di navigazione dell'utente in un'unica riga riassuntiva, identificata univocamente dal session id.

## **4.1 Feature Engineering e Aggregazione**

Per effettuare questa compressione senza perdere il valore informativo dei dati grezzi, abbiamo applicato una tecnica di Feature Engineering guidata da un dizionario logico (`agg_rules`). Invece di avere tanti singoli eventi sparsi, abbiamo costruito delle metriche di sintesi che descrivono il comportamento globale dell'utente durante la sessione. Nello specifico, abbiamo calcolato il volume totale di interazione contando semplicemente il numero di eventi registrati. Abbiamo poi misurato il coinvolgimento temporale sommando tutti i secondi trascorsi sulle varie pagine, e valutato la profondità di esplorazione calcolando la media aritmetica delle percentuali di scorrimento (scroll). Infine, per quanto riguarda il dispositivo utilizzato, abbiamo estratto solo il primo valore registrato all'inizio della sessione, assumendo che l'utente non cambi dispositivo (es. da smartphone a PC) nel mezzo della stessa navigazione.

## **4.2 Definizione della Variabile Target e Prevenzione del Leakage**

Mentre riassumevamo le caratteristiche della sessione, abbiamo calcolato anche la risposta che il nostro modello dovrà imparare a prevedere: la variabile target `cross_municipality`. L'obiettivo era capire se l'utente, durante la sua navigazione, avesse consultato contenuti relativi a un solo comune o a più comuni. Per farlo in modo efficiente a livello distribuito, è stata utilizzata la funzione `nunique()` sulla colonna dei comuni. Questa operazione conta letteralmente quanti comuni diversi compaiono nella stessa sessione: se il conteggio è superiore a uno, la sessione viene etichettata con il valore 1 (navigazione multi-comune), altrimenti con il valore 0. È fondamentale sottolineare che, per evitare il grave errore metodologico noto come Data Leakage, i nomi e i codici esatti dei comuni sono stati rigorosamente scartati dalla tabella finale. Se li avessimo mantenuti, l'algoritmo avrebbe semplicemente "barato", imparando a memoria i comuni anziché generalizzare il comportamento dell'utente.

## **4.3 Pre-elaborazione per il Machine Learning (Pipeline Scikit-Learn)**

L'ultimo passaggio prima dell'addestramento è consistito nel "tradurre" i dati in un formato digeribile dagli algoritmi matematici, utilizzando l'oggetto `ColumnTransformer` di Scikit-Learn. I modelli di classificazione, infatti, presentano due grandi limiti: non sanno leggere il testo e sono sensibili alle scale di grandezza. Per risolvere il problema del testo, la colonna del dispositivo (es. "mobile", "desktop") è stata trasformata usando un `OneHotEncoder`. Questa tecnica converte le categorie in colonne separate composte solo da 0 e 1, permettendo all'algoritmo ad albero di elaborarle senza creare false gerarchie numeriche. Parallelamente, per rispettare i requisiti di scaling richiesti, le variabili numeriche continue (il conteggio degli eventi, la somma dei secondi e la media dello scorrimento) sono state uniformate tramite uno `StandardScaler`. Senza questo passaggio, un algoritmo rischierebbe di dare troppa importanza alla colonna dei secondi (che può arrivare a valori di migliaia) ignorando quella dello scroll (che si ferma a 100), distorcendo così la fase di addestramento.

# **5. Il costo delle operazioni Shuffle-Heavy**

Il fulcro architetturale di questo elaborato coincide con il Focus Tecnico richiesto dalle direttive di progetto, che impone di confrontare i costi computazionali tra una pipeline basata su aggregazioni locali e una pipeline caratterizzata da un'alta intensità di ridistribuzione dei dati sulla rete (shuffle-heavy). Per evidenziare questo fenomeno in modo empirico, lo stress test finale è stato condotto sul dataset esteso (Taglia L, 178.7 MB), monitorando i tempi di esecuzione e profilando l'hardware tramite la Dashboard di Dask.

### 5.1 Pipeline A: Aggregazione Locale e Tree Reduction

La prima strategia (Pipeline A) è stata progettata per minimizzare la comunicazione tra i nodi. Sfruttando la funzione groupby di Dask direttamente sul DataFrame partizionato, il framework ha potuto applicare un paradigma di Tree Reduction. In questa fase, ciascuno dei 4 worker ha eseguito le aggregazioni parziali (sum, mean, count) esclusivamente sulla porzione di dati residente nella propria RAM (circa 23 MB per blocco). Solo in fase finale, i piccoli risultati aggregati sono stati trasmessi al nodo centrale per la fusione definitiva. Questo approccio "silenzioso" sulla rete ha permesso di completare l'intera ingegnerizzazione delle feature in appena 3.7 secondi.

### 5.2 Pipeline B: Rallentamento di rete e Shuffle forzato

La seconda strategia (Pipeline B) ha introdotto deliberatamente un'operazione complessa prima dell'aggregazione: la riassegnazione globale dell'indice tramite il comando set\_index('session\_id'). Nei sistemi distribuiti, l'indice non è un semplice metadato, ma determina fisicamente su quale worker deve risiedere una determinata riga. Per allineare globalmente il dataset secondo le sessioni, Dask è stato costretto a innescare un massiccio Shuffle: i nodi hanno dovuto smontare i propri blocchi di memoria e scambiarsi milioni di frammenti di dati attraverso la rete (o i bus della scheda madre) per raggruppare i record appartenenti alla stessa sessione sul medesimo worker. Il costo di questo traffico dati è stato drastico: la Pipeline B ha impiegato 15.99 secondi, registrando un fattore di rallentamento di 4.32x rispetto all'approccio locale.

### 5.3 Profilazione Visiva tramite Task Stream

L'impatto architetturale dello shuffle è visivamente dimostrabile analizzando la telemetria del cluster.

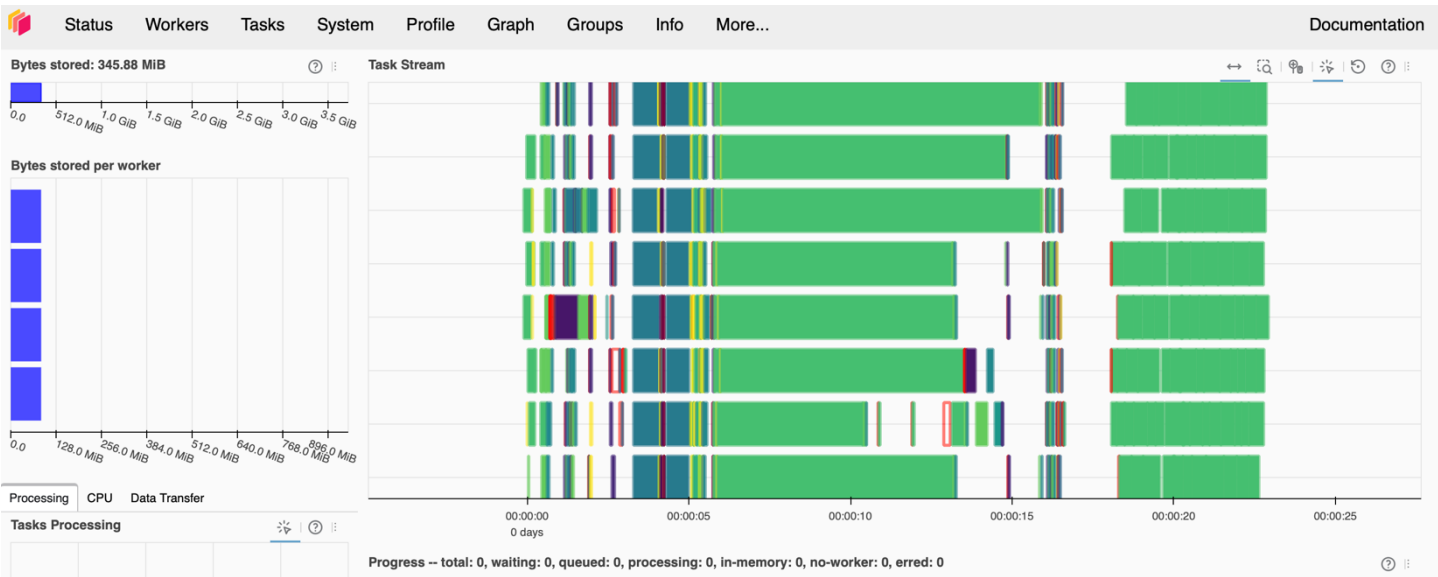


Figura 1: Task Stream della Dashboard Dask durante lo stress test sul Dataset L.

Il grafico evidenzia la totale saturazione degli 8 thread logici a disposizione (4 worker x 2 thread). Nella prima metà dell'esecuzione si notano evidenti frammentazioni (spazi bianchi di Idle time) e segmenti rossi (Data Transfer), firma visiva dell'intenso overhead di rete causato dallo Shuffle della Pipeline B. Nella porzione finale a destra, si osserva invece un blocco di computazione densa e perfettamente parallela, corrispondente all'aggregazione locale e al successivo training del modello Random Forest.

#### 5.4 Validazione dell'Integrità e Non-Determinismo Floating Point

Per garantire che il rimescolamento dei dati non avesse alterato le informazioni, le tabelle prodotte dalle due pipeline sono state sottoposte a un test di uguaglianza. Un primo controllo binario puro (tramite metodo `'equals()'`) ha restituito esito negativo, segnalando una potenziale corruzione. Tuttavia, un'analisi diagnostica approfondita ha rivelato un fenomeno tipico del calcolo parallelo avanzato: il non-determinismo delle operazioni in virgola mobile. In matematica, l'addizione è associativa, ma per le CPU sommare i numeri con la virgola in un ordine diverso genera approssimazioni infinitesimali. Poiché lo shuffle della Pipeline B riordina casualmente i record in ingresso rispetto all'ordine originario letto dalla Pipeline A, le funzioni come la somma dei secondi (`dwel_time_sec`) e la media dello scroll (`scroll_depth`) sono state calcolate su alberi di riduzione differenti. Sostituendo il controllo binario con un calcolo della tolleranza d'errore assoluta, è stato dimostrato che la differenza massima tra le due pipeline ammontava a  $7.27 \times 10^{-12}$  secondi. Essendo l'errore tecnicamente trascurabile (nell'ordine dei millesimi di miliardesimo), si è potuta dichiarare l'assoluta integrità dei dati e la corretta esecuzione di entrambe le architetture, confermando che il rallentamento della Pipeline B era dovuto esclusivamente a inefficienze di rete e non a ricalcoli errati.

## 6. Machine Learning Distribuito e Analisi dei Modelli

L'ultima fase del progetto ha previsto l'addestramento e la valutazione di due algoritmi di tipo Ensemble per la classificazione binaria delle sessioni: Random Forest e AdaBoost. L'obiettivo di questa sezione non è unicamente la ricerca della massimizzazione delle metriche predittive, ma un'analisi critica del comportamento di questi algoritmi quando immersi in un paradigma computazionale distribuito.

### 6.1 Scelta degli Iperparametri e Integrazione con Dask

Per sfruttare l'architettura a 8 core logici configurata in precedenza, l'addestramento dei modelli Scikit-Learn è stato avvolto nel context manager `joblib.parallel_backend('dask')`. Questo strumento fa da ponte tra le librerie, delegando la creazione degli alberi decisionali allo scheduler di Dask affinché il calcolo venga distribuito sui 4 worker locali. Per entrambi i modelli è stato impostato un numero di stimatori base pari a 100 (`n_estimators=100`), un valore ritenuto ottimale per garantire la stabilità statistica dell'Ensemble senza sovraccaricare inutilmente la memoria. Nel caso specifico della Random Forest, è stata applicata un'importante regola di regolarizzazione impostando la profondità massima degli alberi a 5 (`max_depth=5`). In contesti Big Data, lasciare che un albero decisionale cresca senza limiti comporta un rischio elevatissimo di Overfitting (memorizzazione dei dati di training) e un'esplosione incontrollata

della RAM occupata. Limitare la profondità a 5 ha garantito che i worker operassero in totale sicurezza, rispettando il rigido limite di 1 GB di memoria assegnato a ciascuno di essi.

## 6.2 Architettura Algoritmica e Tempi di Esecuzione

L'addestramento sui dati standardizzati della Pipeline A ha rivelato differenze prestazionali sostanziali, dipendenti non dall'hardware, ma dall'architettura matematica intrinseca dei due algoritmi:

**Random Forest (5.23 secondi):** Questo modello si basa sulla tecnica del Bagging. I 100 alberi vengono costruiti in modo totalmente indipendente l'uno dall'altro su sotto-campioni casuali del dataset. Questa indipendenza lo rende un algoritmo Embarrassingly Parallel (perfettamente parallelizzabile): Dask ha potuto assegnare la costruzione di alberi diversi a core diversi simultaneamente, completando il training in poco più di 5 secondi.

**AdaBoost (15.63 secondi):** Ha richiesto un tempo tre volte superiore. AdaBoost si basa sul principio del Boosting, ovvero costruisce gli alberi in modo sequenziale: ogni nuovo albero viene addestrato per correggere gli errori commessi dall'albero immediatamente precedente. Questa rigida dipendenza temporale impedisce al framework di parallelizzare l'addestramento dei singoli alberi, creando un collo di bottiglia architetturale evidente e annullando in parte i vantaggi del cluster Dask.

Figura 2: Output di esecuzione sul dataset Taglia L. Il log evidenzia i tempi di addestramento parallelo (Random Forest vs AdaBoost) e i rispettivi Classification Report. Si nota chiaramente il forte sbilanciamento delle classi nella colonna "support" (147.715 contro 24.292).

```
=== TEMPI DI ESECUZIONE ML ===
Random Forest : 5.23 s
AdaBoost      : 15.63 s

=== REPORT FINALE L ===
RANDOM FOREST:
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 1.00      | 0.96   | 0.98     | 147715  |
| 1            | 0.80      | 0.98   | 0.88     | 24292   |
| accuracy     |           |        | 0.96     | 172007  |
| macro avg    | 0.90      | 0.97   | 0.93     | 172007  |
| weighted avg | 0.97      | 0.96   | 0.96     | 172007  |

```
ADABOOST:
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 1.00      | 0.96   | 0.98     | 147715  |
| 1            | 0.79      | 1.00   | 0.88     | 24292   |
| accuracy     |           |        | 0.96     | 172007  |
| macro avg    | 0.89      | 0.98   | 0.93     | 172007  |
| weighted avg | 0.97      | 0.96   | 0.96     | 172007  |

## 6.3 Analisi Critica delle Metriche Predittive

Dal punto di vista delle performance, entrambi i modelli hanno raggiunto un'accuratezza globale (accuracy) del 96%. Tuttavia, analizzando il Classification Report, è fondamentale notare la forte asimmetria del dataset di validazione, composto da circa 147.000 sessioni mono-comune (Classe 0) contro sole 24.000 sessioni multi-comune (Classe 1). A fronte di questo sbilanciamento, entrambi i modelli si sono comportati in modo eccellente sulla classe maggioritaria (Classe 0), registrando una precision perfetta (1.00) e una recall del 96%. Sulla classe minoritaria target (Classe 1), si osserva invece un interessante trade-off. Entrambi i modelli mostrano una recall altissima (98% per Random Forest, 100% per AdaBoost), a fronte di una precision leggermente inferiore (80% e 79%). Traducendo questi dati nel contesto reale, significa che il modello è estremamente "sensibile": riesce a intercettare praticamente tutte le sessioni multi-comune reali (pochissimi falsi negativi), ma occasionalmente è "troppo zelante", classificando erroneamente come multi-comune alcune sessioni particolarmente lunghe o complesse che in realtà si sono svolte in un singolo comune (falsi positivi). Considerando la parità sostanziale nelle capacità predittive, il verdetto ingegneristico propende nettamente a favore della Random Forest. A parità di efficacia, il suo approccio parallelo (Bagging) si integra in modo nativo ed eccezionalmente più rapido con le architetture di calcolo distribuito come Dask.

## 7. Limiti dell'Analisi e Sviluppi Futuri

Sebbene l'architettura implementata abbia soddisfatto i requisiti progettuali e superato con successo lo stress test, è doveroso evidenziare alcune limitazioni intrinseche al setup sperimentale e delineare le direttrici per eventuali sviluppi futuri.

### 7.1 Limiti Architettureali e di Scala

Il limite principale della simulazione risiede nell'utilizzo di un LocalCluster. Sebbene frammentare artificialmente la memoria a 1 GB per worker abbia permesso di innescare e misurare i colli di bottiglia legati allo Spilling e allo Shuffle, il trasferimento dati è avvenuto sui bus interni della macchina host. In un vero ambiente distribuito (es. nodi AWS o Hadoop), lo scambio di dati via rete TCP/IP introdurrebbe latenze fisiche nettamente superiori, amplificando ulteriormente il divario prestazionale tra la Pipeline A e la Pipeline B misurato in questo elaborato. Inoltre, la dimensione fisica del dataset di stress (Taglia L, ~178 MB), per quanto sufficiente ai fini accademici del test, rappresenta una scala ridotta rispetto ai volumi standard di produzione Big Data (Terabyte o Petabyte), dove inefficienze di rete mal gestite porterebbero al fallimento critico del job.

### 7.2 Limiti Modellistici e Sviluppi Futuri

Dal punto di vista analitico, il modello di Machine Learning ha operato su un dataset fortemente sbilanciato (con un rapporto di circa 6:1 in favore delle sessioni mono-comune). In sviluppi futuri, l'implementazione di tecniche di bilanciamento distribuite, come l'assegnazione di pesi alle classi (Class Weights) o il sottocampionamento della classe maggioritaria, potrebbe migliorare ulteriormente la precisione (precision) nell'individuazione delle sessioni multi-comune. Infine, un'ulteriore evoluzione della pipeline prevedrebbe l'integrazione del catalogo anagrafico (municipality\_catalog.csv), deliberatamente escluso in questa fase per ottimizzare le risorse. Una Join distribuita con tale tabella permetterebbe di estrarre feature geospaziali avanzate (ad esempio, il calcolo della distanza chilometrica tra i diversi comuni visitati o l'appartenenza alla

medesima provincia), arricchendo lo spazio vettoriale e fornendo agli algoritmi ad albero nuove chiavi di interpretazione comportamentale.

## 8. Conclusioni

Il presente elaborato ha dimostrato la progettazione e l'implementazione di una pipeline di elaborazione dati distribuita completa e robusta, sviluppata interamente in ambiente Dask. Partendo dall'ingestione di file log grezzi di dimensioni crescenti (fino alla Taglia L di circa 178 MB), si è adottata una strategia metodica basata sull'over-partitioning per ottimizzare le risorse di un cluster locale fortemente vincolato. Le fasi di pulizia tipologica e di imputazione dei valori nulli hanno preparato il terreno per una rigorosa fase di Feature Engineering, che ha permesso di trasformare dati longitudinali in un dataset strutturato a livello di sessione, privo di Data Leakage rispetto alla variabile target. Il raggiungimento del Focus Tecnico (Sfida S10) ha rappresentato il culmine analitico del progetto: la profilazione visiva e temporale ha dimostrato inequivocabilmente come architetture concettualmente simili possano differire drasticamente in termini di performance. Si è misurato empiricamente come le operazioni che inducono traffico di rete (Shuffle-heavy, come il `set_index`) causino rallentamenti severi (fino a 4.3x) rispetto ad aggregazioni con riduzione ad albero totalmente locali. Infine, l'addestramento parallelo dei modelli di Machine Learning ha consolidato l'importanza della scelta algoritmica nell'ecosistema Big Data. Sebbene sia Random Forest che AdaBoost abbiano ottenuto eccellenti risultati predittivi (96% di accuratezza globale), l'architettura a Bagging della Random Forest si è rivelata nettamente superiore per i paradigmi distribuiti, garantendo tempi di esecuzione frazionari rispetto agli approcci sequenziali. Nel complesso, il progetto ha validato appieno la sinergia tra corretta strutturazione del dato e consapevolezza delle limitazioni hardware in scenari di calcolo parallelo.